

Práctica — API REST con Node.js y Express

Objetivo

Desarrollar una API REST de Personajes aplicando los conceptos de Node.js y Express, organizando el código en rutas, controladores y una fuente de datos separada, implementando las operaciones básicas de un CRUD completo.

Entrega

- **Formato:** Repositorio público de GitHub
- **Qué entregar:** link al repositorio en la tarea de Classroom

Estructura de archivos obligatoria

```
practica-express-[apellido-nombre]/
├── src/
│   ├── controllers/
│   │   └── personajes.controllers.js
│   ├── data/
│   │   └── personajes.js
│   └── routes/
│       └── personajes.routes.js
├── .gitignore
├── app.js
└── package.json
```

Datos iniciales

El archivo `src/data/personajes.js` debe exportar el siguiente arreglo. Está prohibido modificarlo como punto de partida:

```
export const personajes = [  
  { id: 1, nombre: "A-Bomb", imagen:  
    "https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/images/m  
d/1-a-bomb.jpg" },  
  { id: 2, nombre: "Abe Sapien", imagen:  
    "https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/images/m  
d/2-abe-sapien.jpg" },  
  { id: 3, nombre: "Abin Sur", imagen:  
    "https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/images/m  
d/3-abin-sur.jpg" },  
  { id: 4, nombre: "Abomination", imagen:  
    "https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/images/m  
d/4-abomination.jpg" },  
  { id: 5, nombre: "Abraxas", imagen:  
    "https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/images/m  
d/5-abraxas.jpg" },  
];
```

Funcionalidades requeridas

1. Obtener todos los personajes

Implementar un endpoint que devuelva el arreglo completo de personajes en formato JSON.

2. Obtener un personaje por ID

Implementar un endpoint que reciba un ID como parámetro de ruta y devuelva el personaje correspondiente. Si no existe, responder con un código de estado apropiado y un mensaje de error.

Validaciones:

- El ID debe ser un número válido. Si no lo es, responder con error antes de buscar en el arreglo.

3. Agregar un personaje

Implementar un endpoint que reciba nombre e imagen en el cuerpo de la solicitud, genere un nuevo ID automáticamente, agregue el objeto al arreglo y devuelva el personaje creado.

Validaciones:

- Los campos nombre e imagen son obligatorios. Si alguno falta o está vacío, responder con error.

4. Modificar un personaje

Implementar un endpoint que reciba un ID como parámetro de ruta y los datos a actualizar en el cuerpo. Debe modificar el personaje correspondiente en el arreglo y devolverlo actualizado. Si no existe, responder con error.

Validaciones:

- El ID debe ser un número válido.
- El body no puede estar vacío; debe contener al menos un campo.
- Los campos enviados no pueden tener valores vacíos.

5. Eliminar un personaje

Implementar un endpoint que reciba un ID como parámetro de ruta, elimine el personaje del arreglo y confirme la operación. Si no existe, responder con error.

Validaciones:

- El ID debe ser un número válido.

Requisitos técnicos

Estructura y organización

- El archivo app.js solo debe encargarse de configurar la aplicación y montar las rutas
- Las rutas deben definirse en src/routes/personajes.routes.js usando express.Router
- La lógica de cada operación debe implementarse en funciones del archivo src/controllers/personajes.controllers.js
- El arreglo de personajes debe ser la única fuente de datos y residir en src/data/personajes.js

ES Modules

- El proyecto debe configurarse con **"type": "module"** en package.json
- Todos los archivos deben usar la sintaxis import/export de ES Modules
- Está prohibido el uso de require o module.exports

Express

- Uso de `express.json()` como middleware para parsear el cuerpo de las solicitudes
- Uso correcto de los métodos HTTP: GET, POST, PUT y DELETE
- Uso de códigos de estado HTTP apropiados en todas las respuestas

Requisitos de Git y GitHub

Nombre del repositorio: practica-express-[apellido-nombre]

Flujo de trabajo obligatorio:

1. En la rama main inicializar el proyecto: configuración de package.json, app.js base y .gitignore
2. Crear la rama feature/rutas-y-controladores para implementar la estructura de archivos y los endpoints GET, POST, PUT y DELETE
3. Realizar el merge de ambas ramas hacia main antes de la entrega

Commits mínimos: 6 en total entre todas las ramas, siguiendo las convenciones de la cátedra.